

Parte 5

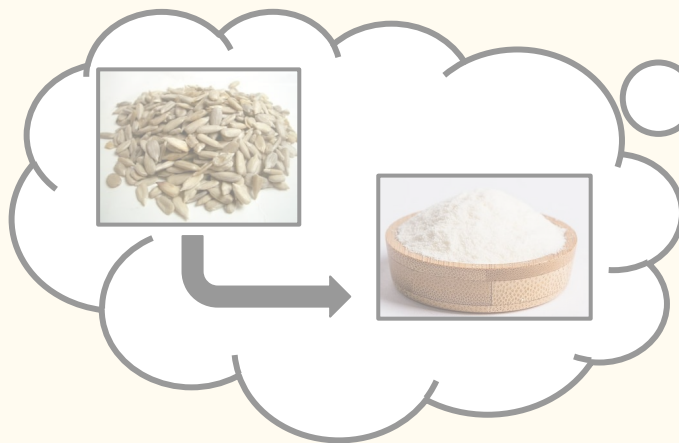
Resolução de Sistemas Lineares - 3

CI1164 - Introdução à Computação Científica

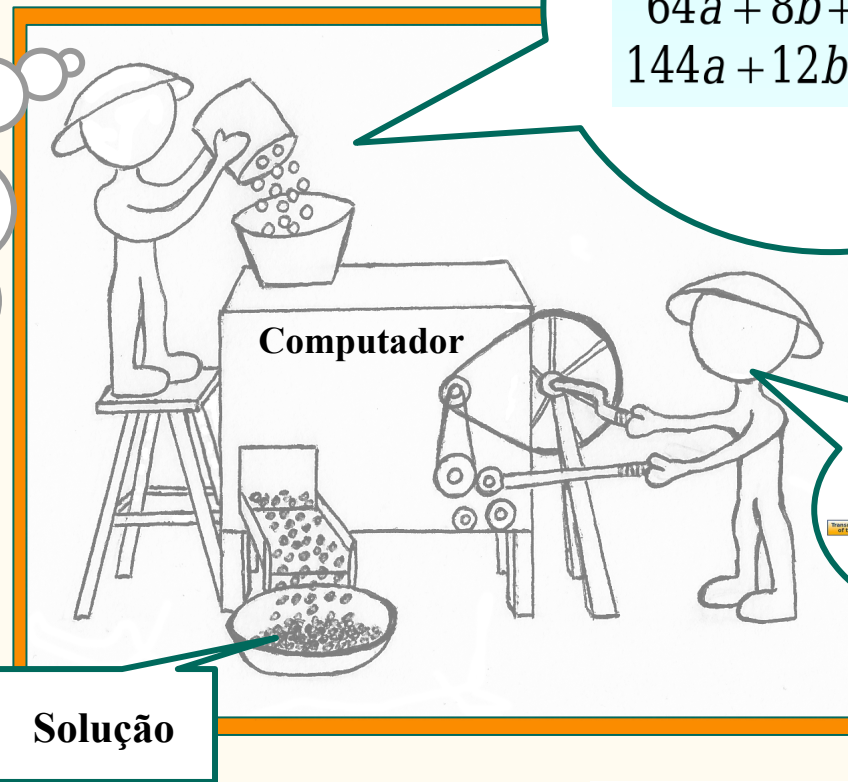
Profs. Armando Delgado e Guilherme Derenievicz

Departamento de Informática - UFPR

Visão geral da disciplina:

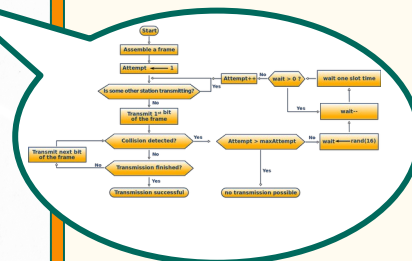


Descrição do Problema



$$\begin{aligned}25a + 5b + c &= 106 \\64a + 8b + c &= 177 \\144a + 12b + c &= 600\end{aligned}$$

Sistemas Lineares



Método Numérico

Solução

Resolução de Sistemas Lineares

$$A = \begin{bmatrix} -\alpha & 0 & 0 & 1 & \alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -\alpha & 0 & -1 & 0 & -\alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 & \alpha & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 & \alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha & 0 & 0 & 1 & \alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & 0 & -1 & 0 & -\alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & \alpha & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 \end{bmatrix}$$

Resolução de Sistemas Lineares

$$A = \begin{bmatrix} -\alpha & 0 & 0 & 1 & \alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -\alpha & 0 & -1 & 0 & -\alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 & \alpha & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 & \alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha & 0 & 0 & 1 & \alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & 0 & -1 & 0 & -\alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & \alpha & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 & 0 \end{bmatrix}$$

Matriz Esparsa

Resolução de Sistemas Lineares

$$A = \begin{bmatrix} -\alpha & 0 & 0 & 1 & \alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -\alpha & 0 & -1 & 0 & -\alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 & \alpha & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 & \alpha & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha & 0 & 0 & 1 & \alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & 0 & -1 & 0 & -\alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & \alpha & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 \end{bmatrix}$$

Matriz Esparsa

Matrizes k-Diagonais

Matriz de Banda

Matriz Esparsa

Matrizes k-Diagonais

$$A = \begin{bmatrix} -\alpha & 0 & 0 & 1 & \alpha & 0 & 0 & 0 & 0 & 0 \\ -\alpha & 0 & -1 & 0 & -\alpha & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 0 & \alpha & 1 \\ 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 & \alpha & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -\alpha \\ 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & \alpha & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\alpha & -1 \end{bmatrix}$$

Matriz de Banda

Matriz Esparsa

$i = j$

Matrizes k-Diagonais

$$i = j+4$$

A =

$-\alpha$	0	0	1	α	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	$-\alpha$	-1	0	0	α	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	α	0	1	0	α	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	-1	$-\alpha$	0	0	1	α	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	-1	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	α	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	-1	$-\alpha$	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	α	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	0

Matriz de Banda

Matriz Esparsa

$$i = j$$

Matrizes k-Diagonais

Matriz de Banda

Matriz Esparsa

$i = j+4$

$i = j+7$

$A =$

$-\alpha$	0	0	1	α	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	$-\alpha$	-1	0	0	α	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	α	0	1	0	α	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	-1	$-\alpha$	0	0	1	α	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	-1	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	α	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	-1	$-\alpha$	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	α	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	0

$i = j$

Matrizes k-Diagonais

Matriz de Banda

$i = j+4$ →

$i = j+7$ →

Se $i > j + 3$
Então $A[i][j] = 0$

Matriz Esparsa

$i = j$

Diagram illustrating a sparse matrix structure with a banded pattern. The matrix is shown with a blue dashed diagonal line and a red solid diagonal line. The matrix is labeled **Matriz de Banda** (Banded Matrix) and **Matriz Esparsa** (Sparse Matrix). The matrix is defined by the condition: **Se** $i > j + 3$ **Então** $A[i][j] = 0$. The matrix is shown with a blue dashed diagonal line and a red solid diagonal line. The matrix is labeled **Matriz de Banda** (Banded Matrix) and **Matriz Esparsa** (Sparse Matrix). The matrix is defined by the condition: **Se** $i > j + 3$ **Então** $A[i][j] = 0$.

Matrizes k-Diagonais

$i = j+4$ →

$i = j+7$ →

Se $i > j + 3$
Então $A[i][j] = 0$

Matriz Esparsa

Se $i < j - 4$
Então $A[i][j] = 0$

Matriz de Banda

$i = j$

$A =$

$-\alpha$	0	0	1	α	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	$-\alpha$	-1	0	0	α	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	α	0	1	0	α	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	-1	$-\alpha$	0	0	1	α	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	-1	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	α	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	-1	$-\alpha$	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	α	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	0	0	0

Matrizes k-Diagonais

$i = j+4$ →

$i = j+7$ →

Matriz de Banda 8

-α	0	0	1	α	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
-α	0	-1	0	-α	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	-α	-1	0	0	α	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	α	0	1	0	α	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	-1	-α	0	0	1	α	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	-α	0	-1	0	-α	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	-1	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	α	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	-1	-α	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	-α	-1	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	α	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	-α	-1	0	0	0	0

Se $i < j - 4$
Então $A[i][j] = 0$

Se $i > j + 3$
Então $A[i][j] = 0$

Matriz Esparsa

$i = j$

Matrizes k-Diagonais

Matriz de Banda $p+q+1$

**Se $i < j - p$
Então $A[i][j] = 0$**

**Se $i > j + q$
Então $A[i][j] = 0$**

Matriz Esparsa

$i = j+4$

$i = j+7$

$i = j$

$A =$

$-\alpha$	0	0	1	α	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	-1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	-1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	$-\alpha$	-1	0	0	α	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	α	0	1	0	α	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	-1	$-\alpha$	0	0	1	α	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	-1	0	-1	0	$-\alpha$	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	-1	0	0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	-1	0	0	0	0	α	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	-1	$-\alpha$	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	α	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	0	0	0	0

Matrizes k-Diagonais

A =

$-\alpha$	0	0	1	α	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	-1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	$-\alpha$	-1	0	0	α	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	α	0	1	0	α	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	-1	$-\alpha$	0	0	1	α	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	$-\alpha$	0	-1	0	$-\alpha$	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	-1	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	α	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	-1	$-\alpha$	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	α	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	$-\alpha$	-1	0	0	0	0	0	0

Matriz de Banda $p+q+1$

Se $i < j - p$
Então $A[i][j] = 0$

Se $i > j + q$
Então $A[i][j] = 0$

Matriz Esparsa

p

Simplificação nos métodos

- Armazenamento dos elementos da matriz em memória
 - Basta as diagonais
- Implementação dos métodos ficam mais simples
 - Menos laços aninhados
 - Menos operações em ponto flutuante

Matrizes Tridiagonais ($p = q = 1$)

$$\begin{bmatrix} d_1 & c_1 & 0 & 0 & 0 \\ a_1 & d_2 & c_2 & 0 & 0 \\ 0 & a_2 & d_3 & c_3 & 0 \\ 0 & 0 & a_3 & d_4 & c_4 \\ 0 & 0 & 0 & a_4 & d_5 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix}$$

Matrizes Tridiagonais (p = q = 1)

$$\begin{bmatrix} d_1 & c_1 & 0 & 0 & 0 \\ a_1 & d_2 & c_2 & 0 & 0 \\ 0 & a_2 & d_3 & c_3 & 0 \\ 0 & 0 & a_3 & d_4 & c_4 \\ 0 & 0 & 0 & a_4 & d_5 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix}$$

```
/* Seja um S.L. de ordem 'n'
*/
void eliminacaoGauss( double **A, double *b, u
    /* para cada linha a partir da primeira */
    for (int i=0; i < n; ++i) {
        for(int k=i+1; k < n; ++k) {
            double m = A[k][i] / A[i][i];
            A[k][i] = 0.0;
            for(int j=i+1; j < n; ++j)
                A[k][j] -= A[i][j] * m;
            b[k] -= b[i] * m;
        }
    }
}
```

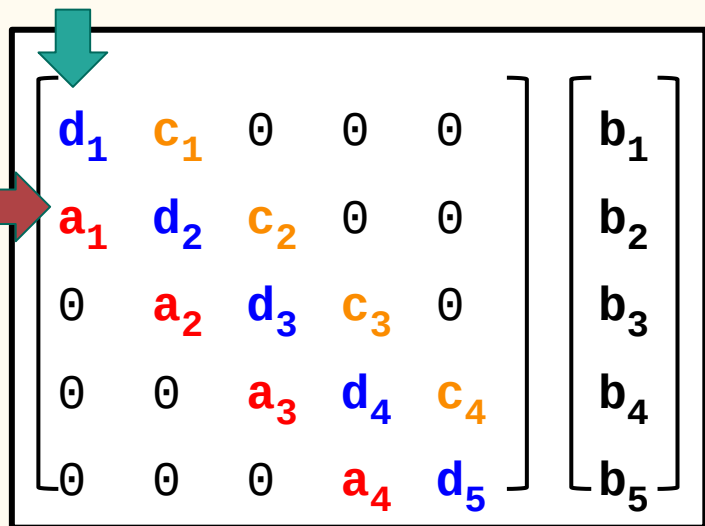
Matrizes Tridiagonais (p = q = 1)


$$\begin{bmatrix} d_1 & c_1 & 0 & 0 & 0 \\ a_1 & d_2 & c_2 & 0 & 0 \\ 0 & a_2 & d_3 & c_3 & 0 \\ 0 & 0 & a_3 & d_4 & c_4 \\ 0 & 0 & 0 & a_4 & d_5 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix}$$



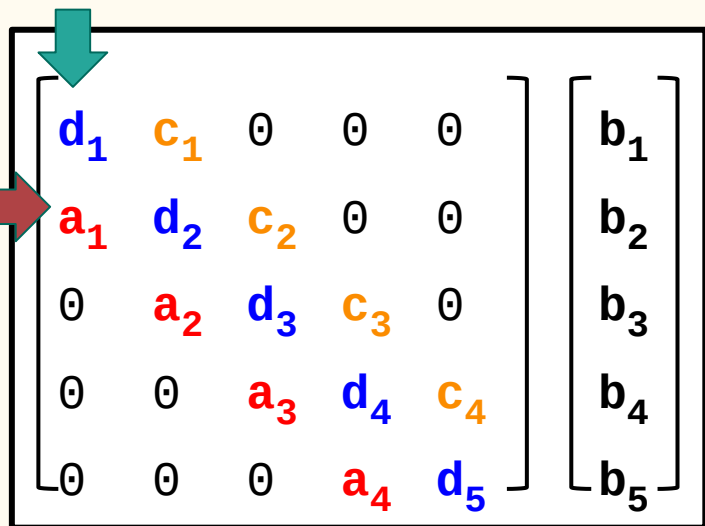
```
/* Seja um S.L. de ordem 'n'
*/
void eliminacaoGauss( double **A, double *b, u
/* para cada linha a partir da primeira */
for (int i=0; i < n; ++i) {
    for(int k=i+1; k < n; ++k) {
        double m = A[k][i] / A[i][i];
        A[k][i] = 0.0;
        for(int j=i+1; j < n; ++j)
            A[k][j] -= A[i][j] * m;
        b[k] -= b[i] * m;
    }
}
```

Matrizes Tridiagonais ($p = q = 1$)


$$\begin{bmatrix} d_1 & c_1 & 0 & 0 & 0 \\ a_1 & d_2 & c_2 & 0 & 0 \\ 0 & a_2 & d_3 & c_3 & 0 \\ 0 & 0 & a_3 & d_4 & c_4 \\ 0 & 0 & 0 & a_4 & d_5 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix}$$

```
/* Seja um S.L. de ordem 'n'
*/
void eliminacaoGauss( double **A, double *b, u
    /* para cada linha a partir da primeira */
    for (int i=0; i < n; ++i) {
        for(int k=i+1; k < n; ++k) {
            double m = A[k][i] / A[i][i];
            A[k][i] = 0.0;
            for(int j=i+1; j < n; ++j)
                A[k][j] -= A[i][j] * m;
            b[k] -= b[i] * m;
        }
    }
}
```

Matrizes Tridiagonais ($p = q = 1$)


$$\begin{bmatrix} d_1 & c_1 & 0 & 0 & 0 \\ a_1 & d_2 & c_2 & 0 & 0 \\ 0 & a_2 & d_3 & c_3 & 0 \\ 0 & 0 & a_3 & d_4 & c_4 \\ 0 & 0 & 0 & a_4 & d_5 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix}$$

```
/* Seja um S.L. de ordem 'n'
*/
void eliminacaoGauss( double **A, double *b, u
/* para cada linha a partir da primeira */
for (int i=0; i < n; ++i) {
for(int k=i+1; k < n; ++k) { k = i+1
    double m = A[k][i] / A[i][i];
    A[k][i] = 0.0;
    for(int j=i+1; j < n; ++j)
        A[k][j] -= A[i][j] * m;
    b[k] -= b[i] * m;
}
}
```

Matrizes Tridiagonais ($p = q = 1$)

$$\begin{bmatrix} d_1 & c_1 & 0 & 0 & 0 \\ a_1 & d_2 & c_2 & 0 & 0 \\ 0 & a_2 & d_3 & c_3 & 0 \\ 0 & 0 & a_3 & d_4 & c_4 \\ 0 & 0 & 0 & a_4 & d_5 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{bmatrix}$$

```
/* Seja um S.L. de ordem 'n'
*/
void eliminacaoGauss( double **A, double *b, u
/* para cada linha a partir da primeira */
for (int i=0; i < n; ++i) {
for(int k=i+1; k < n; ++k) { k = i+1
    double m = A[k][i] / A[i][i];
    A[k][i] = 0.0;
    for(int j=i+1; j < n; ++j)
        A[k][j] -= A[i][j] * m;
    b[k] -= b[i] * m;
}
}
```

Matrizes Tridiagonais ($p = q = 1$)

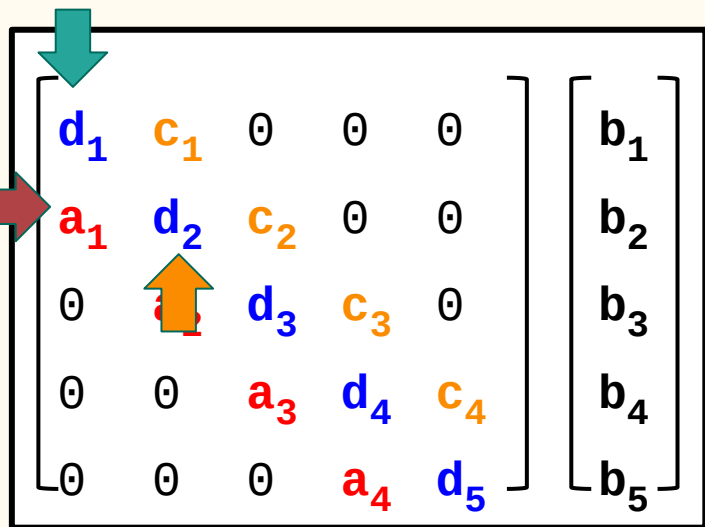
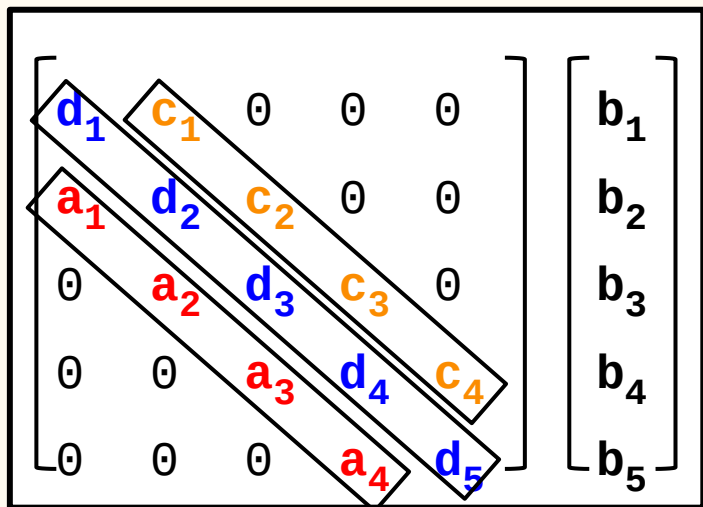


Diagram illustrating a 5x5 tridiagonal matrix system $Ax = b$. The matrix A is shown with elements $d_1, c_1, 0, 0, 0$ in the first row, $a_1, d_2, c_2, 0, 0$ in the second row, $0, a_2, d_3, c_3, 0$ in the third row, $0, 0, a_3, d_4, c_4$ in the fourth row, and $0, 0, 0, a_4, d_5$ in the fifth row. The vector b is shown on the right with elements b_1, b_2, b_3, b_4, b_5 . Arrows indicate the structure: a green arrow points to the diagonal elements d_i , a red arrow points to the sub-diagonal elements a_i , and an orange arrow points to the super-diagonal elements c_i .

```
/* Seja um S.L. de ordem 'n'
*/
void eliminacaoGauss( double **A, double *b, u
/* para cada linha a partir da primeira */
for (int i=0; i < n; ++i) {
    for(int k=i+1; k < n; ++k) { k = i+1
        double m = A[k][i] / A[i][i];
        A[k][i] = 0.0;
        for(int j=i+1; j < n; ++j) j = i+1
            A[k][j] -= A[i][j] * m;
            b[k] -= b[i] * m;
        }
    }
}
```

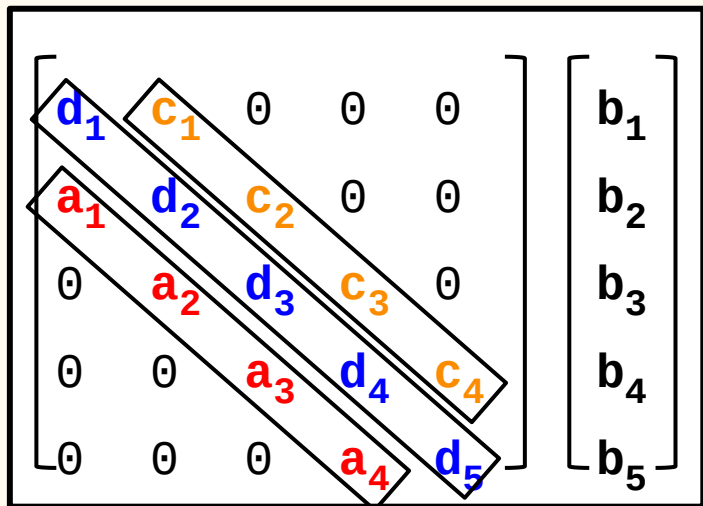

Matrizes Tridiagonais ($p = q = 1$)



```
double b[5];  
double d[5];  
double a[4];  
double c[4];
```

```
/* Seja um S.L. de ordem 'n' */  
void eliminacaoGauss( double **A, double *b, u  
/* para cada linha a partir da primeira */  
for (int i=0; i < n; ++i) {  
    for(int k=i+1; k < n; ++k) { k = i+1  
        double m = A[k][i] / A[i][i];  
        A[k][i] = 0.0;  
        for(int j=i+1; j < n; ++j) j = i+1  
            A[k][j] -= A[i][j] * m;  
        b[k] -= b[i] * m;  
    }  
}
```

Matrizes Tridiagonais ($p = q = 1$)

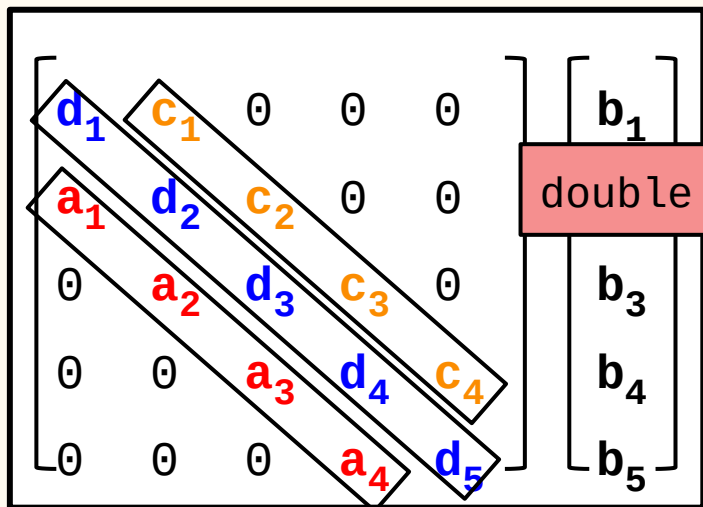


```
double b[5];  
double d[5];  
double a[4];  
double c[4];
```

```
double *d, double *a, double *c
```

```
/* Seja um S.L. de ordem n */  
void eliminacaoGauss( double **A, double *b, unsigned int n )  
{  
    /* para cada linha a partir da primeira */  
    for (int i=0; i < n; ++i) {  
        for(int k=i+1; k < n; ++k) {  
            double m = A[k][i] / A[i][i];  
            A[k][i] = 0.0;  
            for(int j=i+1; j < n; ++j) {  
                A[k][j] -= A[i][j] * m;  
            b[k] -= b[i] * m;  
            }  
        }  
    }  
}
```

Matrizes Tridiagonais ($p = q = 1$)



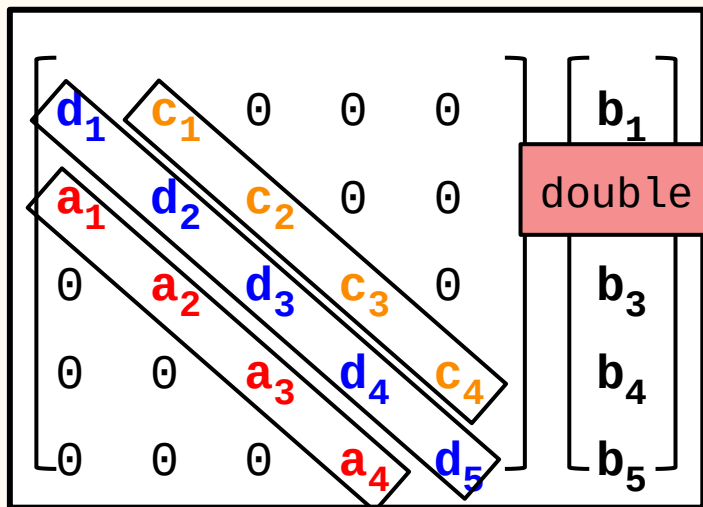
```
double b[5];  
double d[5];  
double a[4];  
double c[4];
```

```
double *d, double *a, double *c
```

```
double m = a[i] / d[i];
```

```
void eliminaGauss( double **A, double *b, u  
/* para cada linha a partir da primeira */  
for (int i=0; i < n; ++i) {  
for(int k=i+1; k < n; ++k) {  
    double m = A[k][i] / A[i][i];  
    A[k][i] = 0.0;  
for(int j=i+1; j < n; ++j)  
        A[k][j] -= A[i][j] * m;  
    b[k] -= b[i] * m;  
}  
}
```

Matrizes Tridiagonais (p = q = 1)



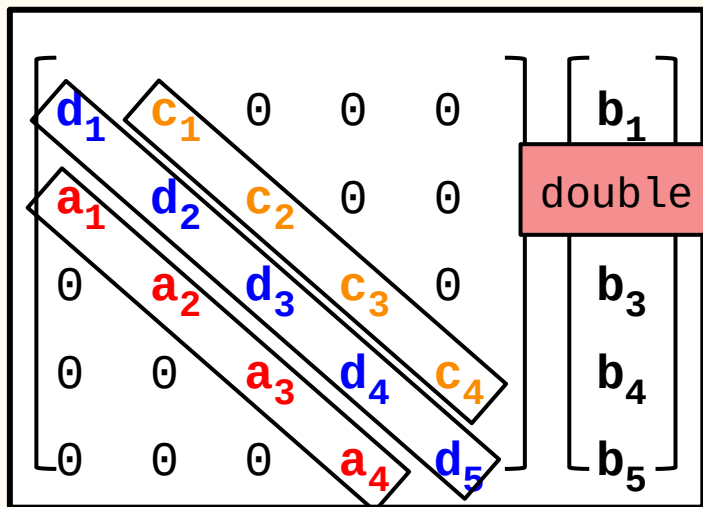
```
double b[5];  
double d[5];  
double a[4];  
double c[4];
```

```
double *d, double *a, double *c
```

```
double m = a[i] / d[i];
```

```
void elimGauss( double **A, double *b, u  
/* para calcular a primeira */  
for (int i=0; i < n; ++i) {  
    a[i] = 0.0;  
    for(int k=i+1; k < n; ++k) {  
        double m = A[k][i] / A[i][i];  
        A[k][i] = 0.0;  
        for(int j=i+1; j < n; ++j)  
            A[k][j] -= A[i][j] * m;  
        b[k] -= b[i] * m;  
    }  
}
```

Matrizes Tridiagonais (p = q = 1)



```
double b[5];
double d[5];
double a[4];
double c[4];
```

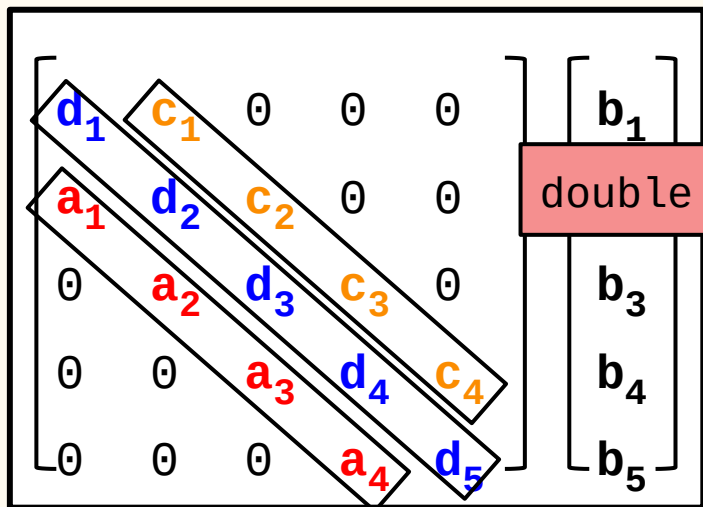
```
double *d, double *a, double *c
```

```
double m = a[i] / d[i];
```

```
void elimGauss( double **A, double *b, u
/* para calcular a primeira */
for (int i=0; i < n; ++i) {
    a[i] = 0.0;
    for(int k=i+1; k < n; ++k) {
        double m = A[k][i] / A[i][i];
        A[k][i] = 0.0;
        for(int j=i+1; j < n; ++j)
            A[k][j] -= A[i][j] * m;
        b[k] -= b[i] * m;
    }
}
```

```
d[i+1] -= c[i]*m;
```

Matrizes Tridiagonais (p = q = 1)



```
double b[5];
double d[5];
double a[4];
double c[4];
```

```
double *d, double *a, double *c
```

```
double m = a[i] / d[i];
```

```
void elimGauss( double **A, double *b, u
/* para calcular a primeira */
for (int i=0; i < n; ++i) {
    a[i] = 0.0;
    for(int k=i+1; k < n; ++k) {
        double m = A[k][i] / A[i][i];
        A[k][i] = 0.0;
        for(int j=i+1; j < n; ++j)
            A[k][j] -= A[i][j] * m;
        b[k] -= b[i] * m;
    }
}
b[i+1] -= b[i]*m;
d[i+1] -= c[i]*m;
```

SL Tridiagonais → Eliminação de Gauss

$$\begin{bmatrix} d_0 & c_0 & 0 & 0 & 0 \\ a_0 & d_1 & c_1 & 0 & 0 \\ 0 & a_1 & d_2 & c_2 & 0 \\ 0 & 0 & a_2 & d_3 & c_3 \\ 0 & 0 & 0 & a_3 & d_4 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

```
double b[5];  
double d[5];  
double a[4];  
double c[4];
```

```
void eliminacaoGauss(double *d, double *a,  
                    double *c, double *b, double *x,  
                    uint n)
```

```
{  
    // TRIANGULARIZAÇÃO: 5(n-1) operações  
    for (int i=0; i < n-1; ++i) {  
        double m = a[ i ] / d[ i ];  
        a[ i ] = 0.0;  
        d[ i+1 ] -= c[ i ] * m;  
        b[ i+1 ] -= b[ i ] * m;  
    }  
    // RETROSUBSTITUIÇÃO: ≈ 3n operações (n grande)  
    x[ n-1 ] = b[ n-1 ] / d[ n-1 ];  
    for (int i=n-2; i >= 0; --i)  
        x[ i ] = (b[ i ] - c[ i ] * x[ i+1 ]) / d[ i ];  
}
```

SL Tridiagonais → Gauss-Seidel

$$\begin{bmatrix} d_0 & c_0 & 0 & 0 & 0 \\ a_0 & d_1 & c_1 & 0 & 0 \\ 0 & a_1 & d_2 & c_2 & 0 \\ 0 & 0 & a_2 & d_3 & c_3 \\ 0 & 0 & 0 & a_3 & d_4 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

```
double b[5];  
double d[5];  
double a[4];  
double c[4];
```

```
void gaussSeidel (double **A,  
                  double *b, double *x, uint n, double tol)  
{  
    double erro = 1.0 + tol;  
    int j, s;  
    while (erro < tol) { // n(2n+1) ≈ 2n² FLOPs / iteração  
        for (int i=0; i < n; ++i) {  
            for (s=0, j=0; j < n; ++j)  
                if (i != j) s += A[i][j] * x[j];  
  
            x[i] = (b[i] - s) / A[i][i];  
  
            // Calcula erro == norma máxima de ( x(k) - x(k-1) )  
            .....  
        }  
    }
```


SL Tridiagonais → Gauss-Seidel

$$\begin{bmatrix} d_0 & c_0 & 0 & 0 & 0 \\ a_0 & d_1 & c_1 & 0 & 0 \\ 0 & a_1 & d_2 & c_2 & 0 \\ 0 & 0 & a_2 & d_3 & c_3 \\ 0 & 0 & 0 & a_3 & d_4 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

```
double b[5];  
double d[5];  
double a[4];  
double c[4];
```

```
void gaussSeidel (double *d, double *a, double *c,  
                  double *b, double *x, uint n, double tol)  
{  
    double erro = 1.0 + tol;  
    while (erro < tol) { // 5(n-2)+6 ≈ 5n FLOPs / iteração  
        x[ 0 ] = (b[ 0 ] - c[ 0 ] * x[ 1 ]) / d[ 0 ];  
  
        for (int i=1; i < n-1; ++i)  
            x[ i ] = (b[ i ] - a[ i-1 ] * x[ i-1 ] - c[ i ] * x[ i+1 ]) / d[ i ];  
  
        x[ n-1 ] = (b[ n-1 ] - a[ n-2 ] * x[ n-2 ]) / d[ n-1 ];  
  
        // Calcula erro == norma máxima de ( x(k) - x(k-1) )  
        .....  
    }
```